

Course
Object-Oriented Programming

Chapter 3:

**Interfaces Graphiques
(GUI Graphical User Interface)**

1

Dr MEKAHLIA Fatma Zohra LAKRID
Associate Professor

MOVEP Laboratory

PLAN

- Généralités sur les interfaces graphiques.
- Composants des interfaces graphiques.
- Les packages AWT et Swing.
- Classes de base.
- Création et affichage d'une fenêtre.
- Placer des composants dans une fenêtre .
- Création de Jar exécutable.
- Gestion des événements.
- Le modèle MVC.

Gestion des événements

GESTION DES ÉVÉNEMENTS

- Le principal objectif d'une application graphique est la programmation événementielle càd l'utilisateur peut déclencher des événements et réagir à ce qui se passe dans la fenêtre.
- Il utilise le clavier et la souris pour intervenir sur le déroulement du programme.
- Le système d'exploitation engendre des événements à partir des actions de l'utilisateur.
- Le programme doit lier des traitements à ces événements.

GESTION DES ÉVÉNEMENTS

- Des événements provoqués par la souris ou le clavier peuvent concerner un composant graphique et le faire réagir. Si on veut que le composant soit sensible à certaines catégories d'événements, il faut le demander explicitement en mettant un objet à l'écoute de l'événement (un Listener) et en indiquant ce qui doit être fait lorsque l'événement survient.

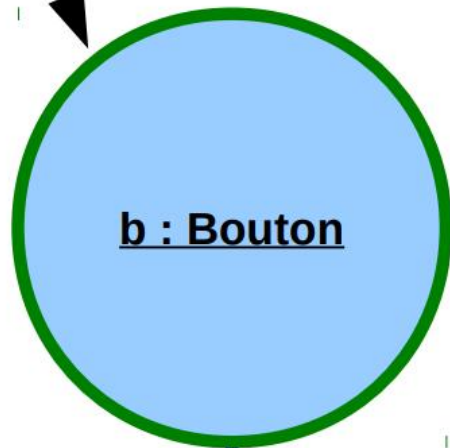
EXEMPLES D'ÉVÉNEMENTS

- Appui sur un bouton de souris ou une touche du clavier.
- Relâchement du bouton de souris ou de la touche.
- Déplacer le pointeur de souris.
- Clic de souris: clic sur un bouton par exemple.
- Choisir un élément dans une liste.
- Modifier le texte d'une zone de saisie.
- Etc,

EVÉNEMENTS / LISTNER

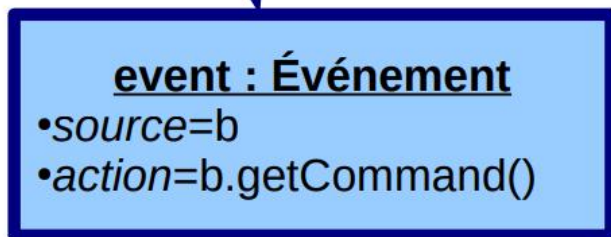
1. Interaction de l'utilisateur
(clic de souris)

Un **événement** est un **objet** créé lorsqu'une action particulière est effectué sur un **objet source**.



new

2. création
d'un évènement



EVÉNEMENTS / LISTNER

- Un composant qui crée des événements est appelé **source** (Exemple: bouton Ajouter de notre calculatrice).
- Le composant **source délègue** le traitement de l'événement au composant **auditeur (listener)**, c'est le composant qui traite l'événement.
- Chacun des composants graphiques a ses écouteurs (*listeners*)

EVÉNEMENTS / LISTNER

- Un composant peut générer des événements et peut avoir plusieurs écouteurs (par exemple, 2 écouteurs pour les clics de souris et un autre pour les frappes de touches du clavier).
- Un écouteur (listener) est une interface qui définit une ou plusieurs méthodes à implémenter et il peut écouter plusieurs composants.

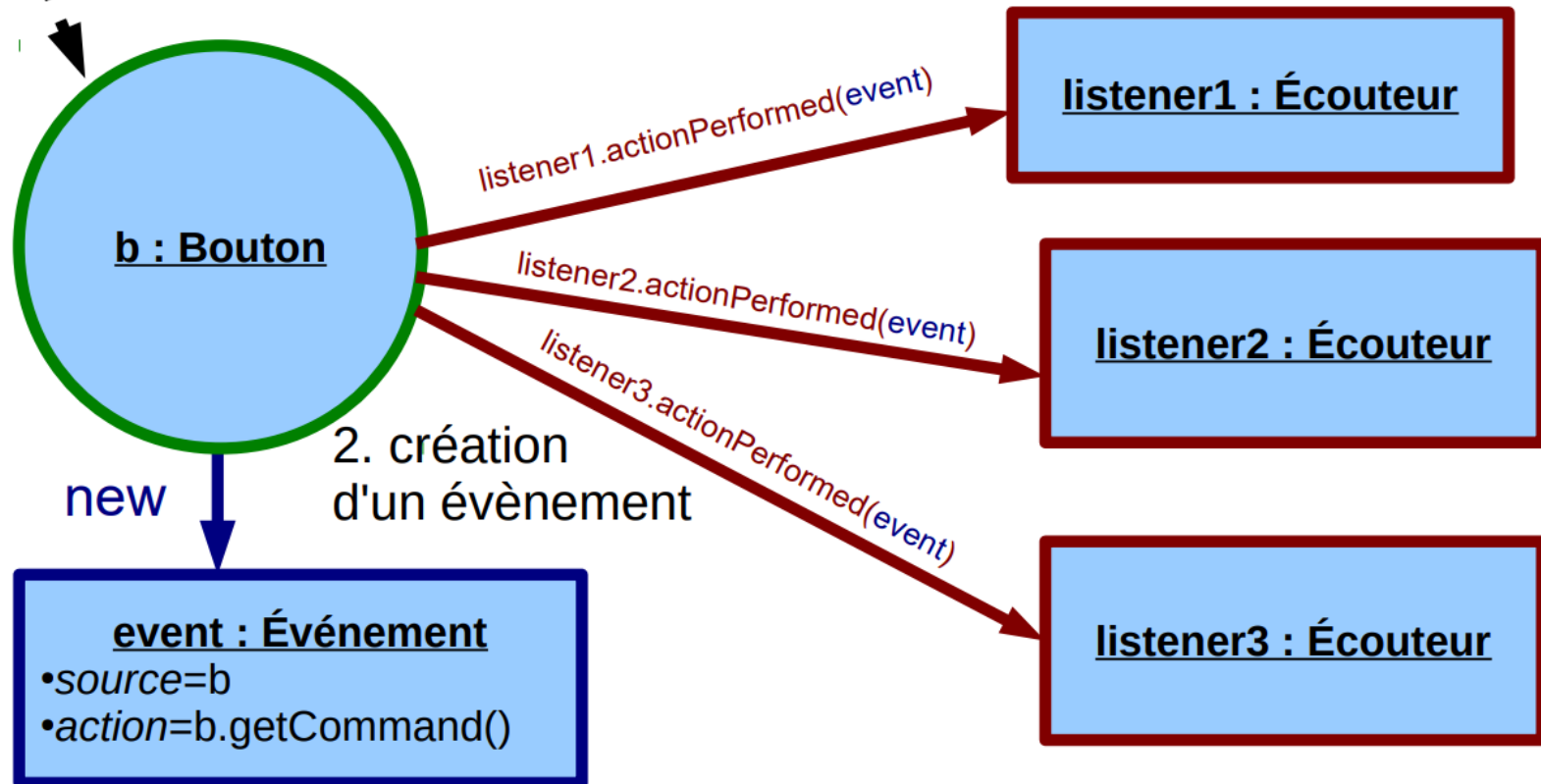
EVÉNEMENTS / LISTNER

- Pour résumer:
- La **gestion des événements en Java**, repose sur un **modèle de délégation d'événements**. L'idée de base est simple : une source d'événement (comme un bouton) génère un événement lorsqu'une action se produit (comme un clic), et un écouteur (listener) s'abonne pour réagir à cet événement.

ÉVÉNEMENTS / LISTNER

1. Interaction de l'utilisateur
(clic de souris)

3. propagation de l'évènement



Un écouteur est un objet réagissant aux évènements d'un objet source.
Pour qu'un objet puisse écouter un objet source, il doit s'enregistrer
auprès de celui ci

EVÉNEMENTS / LISTNER

- **Question:** Quel message sera envoyé par le composant à ses écouteurs pour les prévenir que l'événement qui les intéresse est arrivé ?
- **Réponse :** à chaque type d'écouteur correspond une interface que doit implémenter la classe de l'écouteur .

EVÉNEMENTS / LISTNER

- Tout **événement** hérite de la classe **EventObject**. Les différents types d'événements sont représentés par des classes différentes
- Tout **listner** correspond à une interface qui hérite de **EventListener**.
- Toute classe désirant recevoir des notifications d'un type d'événement donné devra implémenter l'interface correspondante :
- Et toutes les classe héritent de la classe **java.util.EventObject**.

LES CLASSES D'ÉVÉNEMENTS (EVENT CLASSES)

Classe	Description
ActionEvent	Action simple (ex : clic sur un bouton)
ItemEvent	Changement d'état d'un élément (checkbox, choix dans une liste)
AdjustmentEvent	Changement sur une scrollbar
TextEvent	Texte modifié dans un champ
FocusEvent	Gagner ou perdre le focus
KeyEvent	Touche pressée/claviée
MouseEvent	Mouvement/clic souris
MouseWheelEvent	Roue de souris tournée
WindowEvent	Actions sur les fenêtres (ouverture, fermeture, etc.)
ComponentEvent	Changement de taille, de position d'un composant
ContainerEvent	Ajout/retrait d'un composant dans un container

LES INTERFACES D'ÉCOUTEURS (LISTENER INTERFACES)

Événement associé	Interfaces	Méthodes principales
ActionEvent	ActionListener	actionPerformed(ActionEvent e)
ItemEvent	ItemListener	itemStateChanged(ItemEvent e)
AdjustmentEvent	AdjustmentListener	adjustmentValueChanged(...)
TextEvent	TextListener	textValueChanged(...)
FocusEvent	FocusListener	focusGained(...), focusLost(...)
KeyEvent	KeyListener	keyPressed(...), keyReleased(...), keyTyped(...)
MouseEvent	MouseListener, MouseMotionListener	mouseClicked(...), mousePressed(...),mouseDragged(...),

LES INTERFACES D'ÉCOUTEURS (LISTENER INTERFACES)

Événement associé	Interfaces	Méthodes principales
MouseEvent	MouseListener	mouseWheelMoved(...)
WindowEvent	WindowListener, WindowFocusListener, WindowStateListener	windowOpened(...), windowClosing(...), windowGainedFocus(...), windowLostFocus(...), windowStateChanged(...)
ComponentEvent	ComponentListener	componentResized(...)
ContainerEvent	ContainerListener	componentAdded(...), componentRemoved(...)

CLASSES D'ÉVÉNEMENTS / LISTNER

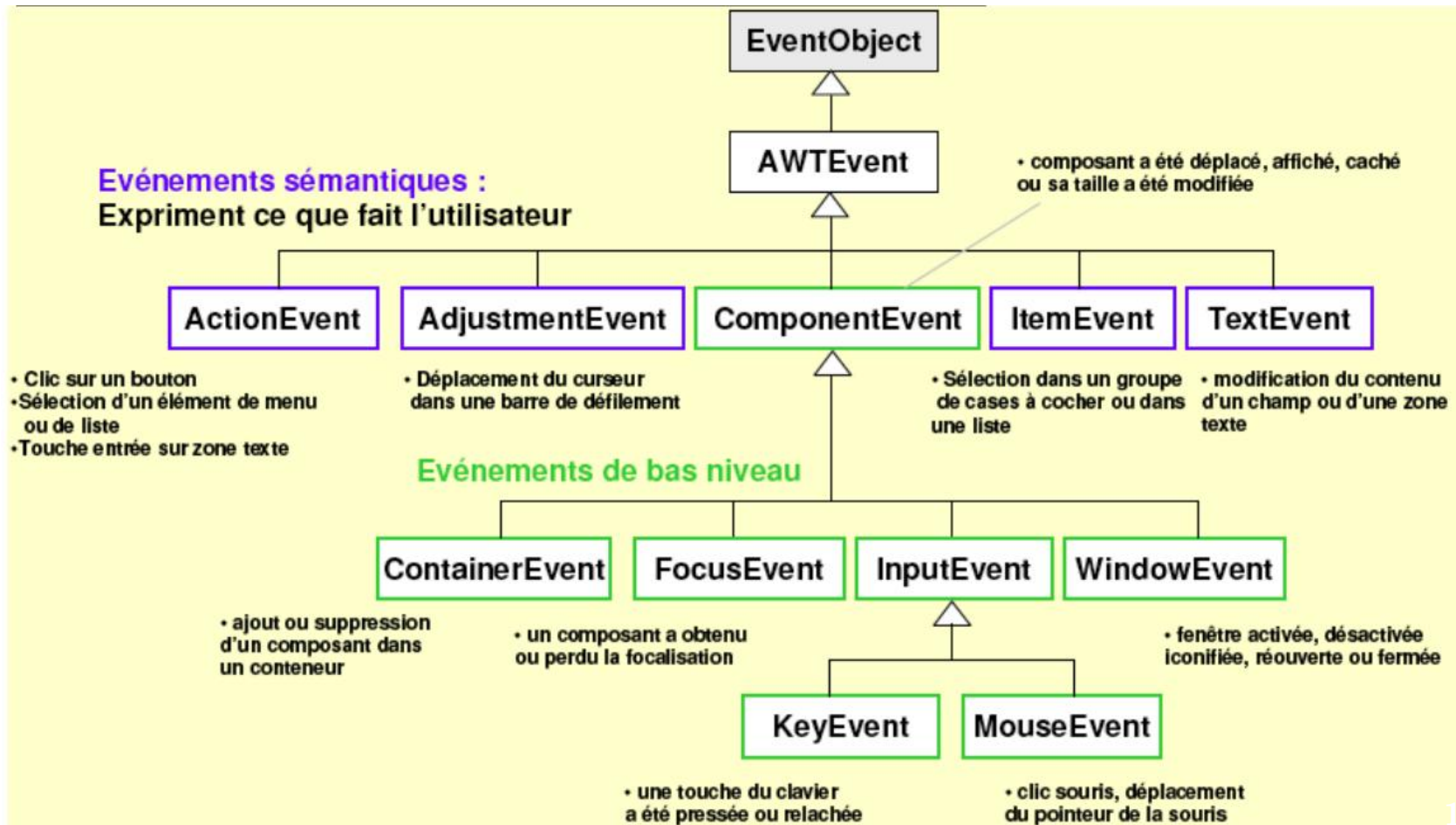
○ Les sources :

- Boutons : JButton, JRadioButton, JCheckBox, JToggleButton
- Menus : JMenuItem, JMenu, JRadioButtonMenuItem, JCheckBoxMenuItem
- Texte : JTextField
- ...

○ Les listeners:

- Il faut implémenter l'interface qui correspond au type de l'événement !!

UNE PARTIE DE LA HIÉRARCHIE DES ÉVÉNEMENTS AWT



CLASSE : ACTIONEVENT

- Cette classe décrit des événements qui vont le plus souvent déclencher un traitement (une *action*):
 - clic sur un bouton.
 - return dans une zone de saisie de texte.
 - choix dans un menu.
- Ces événements sont très fréquemment utilisés et ils sont très simples à traiter.

INTERFACE: **ACTION**LISTENER

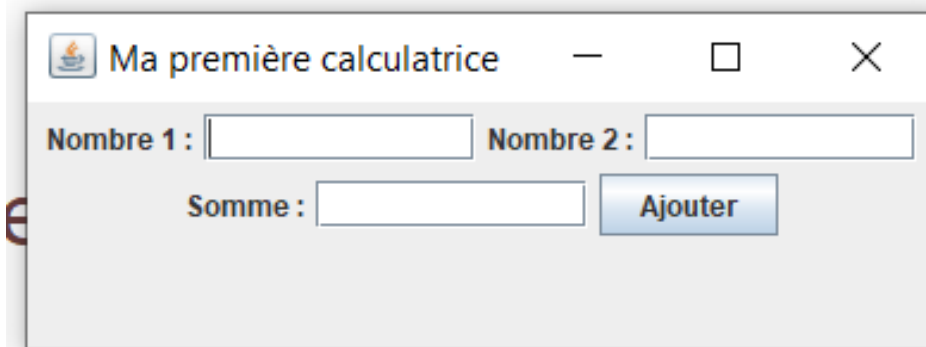
- Un objet *ecouteur intéressé par les événements* de type « action » (classe **ActionEvent**) doit appartenir à une classe qui implémente l'interface **java.awt.event.ActionListener**

EXEMPLES D'UTILISATION : BOUTON

- Exemple d'un **bouton** :
 - Un bouton est un élément graphique sur lequel l'utilisateur peut cliquer pour déclencher une action.
 - Le bouton ne fait rien tant que l'utilisateur n'a pas cliqué dessus.
 - Lors d'un clique un événement est créé ... reste à le traiter !

ECOUTER UN BOUTON

- Le bouton « **Ajouter** » de notre calculatrice de TP4 (partie 1) n'été pas encore prêt à réagir afin d'additionner les deux nombres (voir projet CalculatriceVer1).



ECOUTER UN BOUTON

- Si on veut que le bouton **Ajouter** soit sensible au clic et affiche la somme des deux nombres, il faut le demander explicitement en **ajoutant un objet à l'écoute de l'événement** (un Listener) et en indiquant ce qui doit être fait lorsque l'événement survient.

COMMENT ACTIVER LE BOUTON AJOUTER

- Créer une sous-classe qui implémente l'interface **ActionListener**.
- Implémenter la seule méthode abstraite de l'interface **ActionListener** (**actionPerformed**).
- L'implémentation de la méthode **actionPerformed** (**ActionEvent e**) sera exécuté suite au clic sur le bouton.
- Faire une liaison entre notre bouton **Ajouter** et l'objet de la sous-classe créée, en utilisant la méthode **addActionListener (Object o)** sur le bouton.

SOUS-CLASSE : EVENTAJOUTER IMPLEMENTS ACTIONLISTENER

```
o  /****Sous-classe***/
o  class EventAjouter implements ActionListener{
o  @Override
o  public void actionPerformed(ActionEvent arg0) {
o  // TODO Auto-generated method stub
o  try{
o  int num1, num2, somme;
o  num1=Integer.parseInt(entrée1.getText());
o  num2=Integer.parseInt(entrée2.getText());

o  somme=num1+num2;
o  resultat.setText(Integer.toString(somme));
o  }
o  catch (Exception e){
o  JOptionPane.showMessageDialog(null, "entrer un nombre");}
o  }}
o  /**Liaison entre le bouton Ajouter et l'objet de la sous-classe**/
o  lancer.addActionListener(new EventAjouter());
```

APPROCHES POSSIBLES POUR LES LISTENERS

- Nous pouvons ajouter un listener :
- **via une classe interne,**
- **via une classe externe,** ou autre techniques

CYCLE DE GESTION D'ÉVÉNEMENT

1. L'utilisateur effectue une action (ex : clic).
2. L'objet Swing génère un événement (ActionEvent, MouseEvent, etc.).
3. L'événement est envoyé à tous les objets ayant implémenté l'écouteur correspondant.
4. Le code dans actionPerformed, etc., s'exécute.

INTERFACE: MOUSELISTENER

- L'interface `MouseListener` prend en compte:
- L'appui ou le relâchement du bouton de la souris dans l'espace du composant,
- l'entrée ou la sortie du curseur de la souris de cet espace, ou un clic sur le composant.
- A noter qu'un clic génère aussi les événements appui et relâchement.
- Cinq méthodes de l'interface `MouseListener` traitent ces 5 événements :

INTERFACE: MOUSELISTENER

- void mouseClicked (MouseEvent evt) : clic sur l'espace du composant.
- void mouseEntered (MouseEvent evt) : le curseur entre dans l'espace du composant.
- void mouseExited (MouseEvent evt) : le curseur sort de l'espace du composant.
- void mousePressed (MouseEvent evt) : bouton de la souris appuyé sur le composant.
- void mouseReleased (MouseEvent evt) : bouton de la souris relâché sur le composant.

INTERFACE: MOUSELISTENER

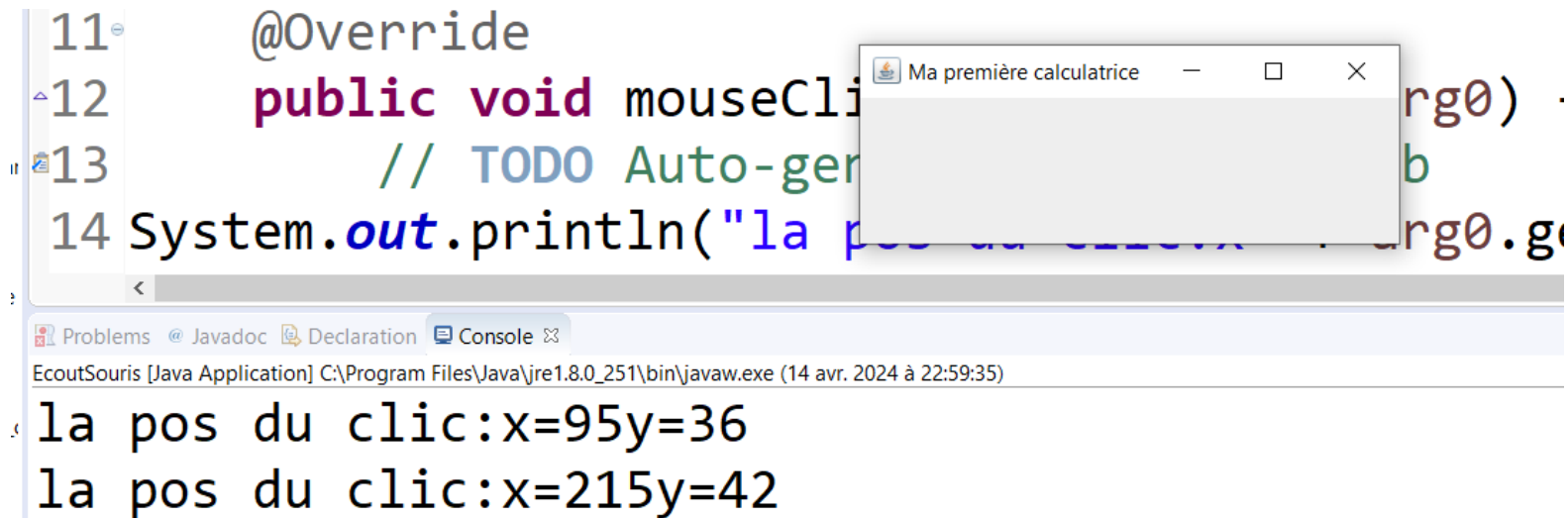
- `/** Un programme qui permet d'afficher la position (X,Y) de clic souris dans le panel */`
- `import java.awt.event.MouseEvent;`
- `import java.awt.event.MouseListener;`
- `import javax.swing.JFrame;`
- `import javax.swing.JPanel;`
- `public class EcoutSouris implements MouseListener {`
- `@Override`
- `public void mouseClicked(MouseEvent arg0) {`
- `// TODO Auto-generated method stub`
- `System.out.println("la pos du clic : x="+arg0.getX()+"y="+arg0.getY());`
- `}`
- `@Override`
- `... etc`

INTERFACE: MOUSELISTENER

- `public static void main (String [] args){`
- `JFrame fenetre= new JFrame("Ma première calculatrice");`
- `fenetre.setSize(400,150);`
- `fenetre.setVisible(true);`
- `fenetre.setLocationRelativeTo(null);`
- `JPanel p = new JPanel ();`
- `fenetre.add(p);`
- `EcoutSouris ecout = new EcoutSouris ();`
- `p.addMouseListener (ecout);`
- `}`

INTERFACE: MOUSELISTENER

```
11 @Override
12 public void mouseClicked(MouseEvent e) {
13     // TODO Auto-generated method stub
14     System.out.println("la pos du clic:x=" + e.getX() + "y=" + e.getY());
15 }
```



la pos du clic:x=95y=36
la pos du clic:x=215y=42

L'INTERFACE MOUSEMOTIONLISTENER

- MouseMotionListener prend en compte les déplacements de la souris quand le curseur se trouve dans l'espace du composant ; il gère deux événements : le déplacement du curseur sans appui sur le bouton, et le déplacement avec bouton de la souris constamment appuyé. Deux méthodes de l'interface MouseMotionListener traitent ces 2 événements :
- **void mouseMoved (MouseEvent evt) :** survol du composant, bouton non appuyé.
- **void mouseDragged (MouseEvent evt) :** survol du composant, bouton appuyé.

Merci